

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Activity / Exp No.: Project-Based Learning (PBL) Part 1

Student Name: J.Srikar

Roll No.: A24126552083

Date: 21-09-2026

1. TITLE

Project-Based Learning (PBL) part 1

Object-Oriented Programming in JavaScript Using ES6 Classes and Constructor Methods

2. AIM

To design, implement, and verify an Object-Oriented JavaScript program utilizing ES6 class syntax, constructor initialization, instance properties, and prototype member methods to encapsulate, instantiate, and display student profile records in a Node.js runtime environment.

3. OBJECTIVE

- To declare and structure reusable object blueprints using the ES6 class syntax (class Student).
- To initialize unique instance variables (name, age, course) via the constructor() lifecycle method with parameter binding.
- To utilize the this contextual execution keyword to bind state properties directly to the newly allocated instance object.
- To define shared instance member methods (displayDetails()) encapsulated on the class prototype for optimal memory utilization.
- To instantiate multiple distinct student entities (student1, student2, student3) using the new operator.
- To invoke instance member methods sequentially and observe proper state retention and isolation across instances.
- To verify console output formatting, string concatenation, and command-line execution in the Node.js V8 runtime environment.

4. THEORY

Object-Oriented Programming (OOP) is a software design paradigm organized around real-world entities termed objects—data structures containing attributes (properties) and executable behaviors (methods). Traditional object-oriented languages such as Java and C++ utilize class-based architectures with rigid type systems. In contrast, JavaScript was originally architected on Prototypal Inheritance, where objects inherit directly from other objects via internal prototype linkages ([[Prototype]]).

With the release of ECMAScript 2015 (ES6), JavaScript introduced formal class syntax as syntactic sugar over existing constructor functions and prototype chaining. This modern abstraction simplifies object encapsulation, enhances code readability, and provides familiar syntax for developers from class-based backgrounds while preserving JavaScript's dynamic delegation model.

The constructor method is a special lifecycle method invoked automatically whenever an object is instantiated with the 'new' keyword. When 'new Student(...)' executes, the JavaScript V8 engine performs four critical operations: (1) It allocates a new, empty object in heap memory; (2) It links the new object's internal prototype (`__proto__`) to `Student.prototype`; (3) It binds the execution context identifier 'this' to the new object and executes constructor statements; (4) It automatically returns the initialized object instance.

Member methods defined inside the class body, such as `displayDetails()`, are placed onto `Student.prototype` rather than being re-created in each instance's local memory footprint. When `student1.displayDetails()` is called, the JavaScript engine traverses the prototype chain to resolve the method and binds 'this' to `student1`, ensuring efficient memory reuse across all student instances.

5. ALGORITHM / PROCEDURE

1. Open a terminal and navigate to the project directory (PBL/JavaScript).
2. Create the JavaScript source file named `student.js`.
3. Declare the blueprint class named `Student` using the ES6 class keyword.
4. Implement the constructor method `constructor(name, age, course)` accepting student attributes as formal arguments.
5. Assign the argument values to instance properties using `this.name = name`, `this.age = age`, and `this.course = course`.
6. Define an instance member method named `displayDetails()` inside the class body without using the function keyword.
7. Inside `displayDetails()`, format and log the student's name, age, course, and a horizontal divider line ('-----') using `console.log()`.
8. Instantiate the first student object `student1` with arguments ('Rahul', 20, 'JavaScript') using the new operator.
9. Instantiate the second student object `student2` with arguments ('Priya', 21, 'Python') using the new operator.
10. Instantiate the third student object `student3` with arguments ('Arun', 19, 'Java') using the new operator.
11. Invoke `student1.displayDetails()` to print Rahul's profile to the standard output.
12. Invoke `student2.displayDetails()` to print Priya's profile to the standard output.
13. Invoke `student3.displayDetails()` to print Arun's profile to the standard output.
14. Execute the program in the terminal using the command `node student.js`.
15. Verify terminal output formatting, proper variable interpolation, and state isolation across all three student instances.

6. PROGRAM

`student.js`

```
// Define a class
class Student {

    // Constructor initializes object properties
    constructor(name, age, course) {
        this.name = name;
        this.age = age;
        this.course = course;
    }

    // Method to display student details
    displayDetails() {
        console.log("Name: " + this.name);
        console.log("Age: " + this.age);
        console.log("Course: " + this.course);
        console.log("-----");
    }
}

// Create multiple objects from the Student class
let student1 = new Student("Rahul", 20, "JavaScript");
let student2 = new Student("Priya", 21, "Python");
let student3 = new Student("Arun", 19, "Java");

// Call the common method for each object
student1.displayDetails();
student2.displayDetails();
student3.displayDetails();
```

7. CODE EXPLANATION

Code Snippet	Detailed Technical Explanation
<code>class Student { ... }</code>	Declares an ES6 class blueprint encapsulating properties and methods for student profile instances.
<code>constructor(name, age, course)</code>	Special lifecycle method invoked upon instantiation to initialize object attributes with passed arguments.
<code>this.name = name;</code>	Binds the passed 'name' string parameter directly to the newly allocated instance object's property.
<code>this.age = age;</code>	Assigns the numeric 'age' parameter value to the active object instance state.
<code>this.course = course;</code>	Assigns the specified 'course' string parameter value to the active object instance state.
<code>displayDetails() { ... }</code>	Defines an instance member method residing on Student.prototype for efficient shared memory execution.
<code>console.log("Name: " + this.name);</code>	Concatenates label with instance property this.name and prints the formatted string to stdout.

<code>let student1 = new Student(...);</code>	Invokes the constructor via 'new', allocating heap memory and binding properties for Rahul.
<code>let student2 = new ...; student3 = ...;</code>	Constructs separate distinct instances in memory (Priya, Arun) demonstrating independent state isolation.
<code>student1.displayDetails();</code>	Invokes the prototype method with 'this' context bound to student1, printing formatted record.

8. TEST CASES AND EXECUTION DATA

Test Case	Target	Input / Action	Description	Expected Result	Actual Result	Status
TC-01	student1	new Student("Rahul", 20, "JavaScript")	Instantiate 1st student object	Properties initialized: Rahul, 20, JavaScript	Object created; properties stored accurately	Pass
TC-02	student2	new Student("Priya", 21, "Python")	Instantiate 2nd student object	Properties initialized: Priya, 21, Python	Object created; student1 state untouched	Pass
TC-03	student3	new Student("Arun", 19, "Java")	Instantiate 3rd student object	Properties initialized: Arun, 19, Java	Object created with distinct memory space	Pass
TC-04	Methods	student1.displayDetails()	Execute member method on instance 1	Formatted record printed with divider line	Printed: Name: Rahul, Age: 20, Course: JS	Pass
TC-05	Prototype	st1.display === st2.display	Verify shared prototype method reference	Both reference Student.prototype method	Evaluates true; zero memory duplication	Pass

9. OUTPUT

Output 1: Execution Event & Console Trace

```
[Event 1] Initialized JavaScript execution via command: node student.js
[Event 2] Compiled class Student blueprint and registered prototype methods.
[Event 3] Instantiated student1: { name: 'Rahul', age: 20, course: 'JavaScript' }
[Event 4] Instantiated student2: { name: 'Priya', age: 21, course: 'Python' }
[Event 5] Instantiated student3: { name: 'Arun', age: 19, course: 'Java' }
[Event 6] Executed student1.displayDetails() -> Logged profile with delimiter.
[Event 7] Executed student2.displayDetails() -> Logged profile with delimiter.
[Event 8] Executed student3.displayDetails() -> Logged profile with delimiter.
[Event 9] Process exited with status code 0 (Execution Success).
```

Output 2: Terminal Execution Output of node student.js

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
jsrikar@Js-MacBook-Air JavaScript % node student.js
Name: Rahul
Age: 20
Course: JavaScript
-----
Name: Priya
Age: 21
Course: Python
-----
Name: Arun
Age: 19
Course: Java
-----
jsrikar@Js-MacBook-Air JavaScript %
```

Ln 28, Col 27 Spaces: 4

10. RESULT

Thus, an Object-Oriented JavaScript program utilizing ES6 class architecture, constructor parameter binding, instance member method encapsulation, and prototypal inheritance was successfully designed, implemented, and verified in Node.js across all test cases with accurate console output.